

2ourc3's - Baby Keygen 2

vendredi 8 mai 2026 02:15

<https://crackmes.one/crackme/639db6c133c5d43ab4ecfecf>

2ourc3's Baby Keygen 2

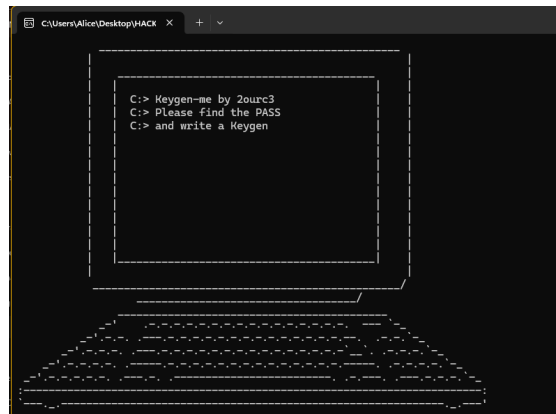
Author: 2ourc3	Language: C/C++	Upload: 2022-12-17 12:32	Download
Platform: Windows	Difficulty: 1.6	Quality: 4.2	
Downloads: 133	Size: 76.11 KB	Writeups: 3	Comments: 9

Description

This challenge is quite easy, find the valid key to get the pass, and write a small, simple, keygen to generate other valid key.

Made by 2ourc3 (<https://www.bushido-sec.com/>)

Comments (9) Writeups (3)



Le but est de trouver la génération de clé et en coder une :

```
5-keygen-me-zust-abit-more-difficult.exe - PID: 25824 - Module: 5-keygen-me-zust-abit-more-difficult.exe - Thread: Main Thread 38600 - x64dbg
File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles i3 Trace
000000000040160E 55 push rbp
000000000040160F 48 89E5 mov rbp, rsp
0000000000401612 48 83EC 30 sub rsp, 30
0000000000401616 E8 9D100000 call 5-keygen-me-zust-abit-more-difficult
0000000000401618 E8 30FFFFFF call 5-keygen-me-zust-abit-more-difficult
0000000000401620 C745 EC 0000 mov dword ptr ss:[rbp-14], 0
0000000000401627 C745 E8 0A00 mov dword ptr ss:[rbp-18], A
0000000000401629 48 8D0D F03 lea rcx, qword ptr ds:[404725]
0000000000401632 E8 C1500000 call <MP.&printf>
000000000040163A 48 8D45 DA lea rax, qword ptr ss:[rbp-26]
000000000040163E 48 89C1 mov rcx, rax
0000000000401641 E8 DA500000 call <MP.&gets>
0000000000401646 48 8D45 DA lea rax, qword ptr ss:[rbp-26]
000000000040164A 48 89C1 mov rcx, rax
000000000040164D E8 9E150000 call <MP.&strlen>
0000000000401652 8945 E4 mov dword ptr ss:[rbp-1C], eax
0000000000401655 837D E4 09 cmp dword ptr ss:[rbp-1C], 9
0000000000401658 7E 3D jle 5-keygen-me-zust-abit-more-difficult
000000000040165B 837D E4 03 jle 5-keygen-me-zust-abit-more-difficult
000000000040165F 0B845 DA movzx eax, byte ptr ss:[rbp-26]
0000000000401663 C7 74 cmp al, 74
0000000000401667 7E 37 jle 5-keygen-me-zust-abit-more-difficult
0000000000401669 0B845 DE movzx eax, byte ptr ss:[rbp-22]
000000000040166D 3C 2D cmp al, 2D
0000000000401671 7E 3F jle 5-keygen-me-zust-abit-more-difficult
0000000000401676 E8 F5FFFFFF call 5-keygen-me-zust-abit-more-difficult
000000000040167E 48 8945 F0 mov qword ptr ss:[rbp-10], rax
0000000000401682 8B45 F8 mov edx, dword ptr ss:[rbp-18]
0000000000401687 41 890D mov r9d, edx
000000000040168C 48 89C7 mov rcx, rax
000000000040168F 48 8D0D B23 lea rcx, qword ptr ds:[404740]
0000000000401693 E8 75150000 call <MP.&printf>
0000000000401698 48 8945 F0 mov qword ptr ss:[rbp-10], rax
000000000040169D E8 90000000 call 5-keygen-me-zust-abit-more-difficult
00000000004016A0 48 8D0D B53 lea rcx, qword ptr ds:[40475C]
00000000004016A7 E8 5C150000 call <MP.&printf>
0A: '\n'
0000000000404725: "Enter a your license key: "
=> rax pointe sur notre string
=> comparaison avec la taille
=> on recupere la taille de notre string dans EAX
=> jmp bad boy si le code fait moins de 9 caractere ou plus
=> comparaison bad boy si le code fait que 3 caractere ou moins
=> jmp bad boy si le code fait que 3 caractere ou moins
=> met dans EAX le 1er byte de notre input qui se situe dans le stack
74: 't'
=> jmp bad boy si le 1er byte de notre input est different de t
2D: '-'
=> jmp bad boy si le 5er byte de notre input est different de -
=> call a verifier
[rbp-10]: "BabyKeygen"
=> met dans EDI la taille du key cache (BabyKeygen)
[rbp-10]: "BabyKeygen"
0000000000404740: "KEY VALID: The pass is %s%"
000000000040475C: "INVALID KEY"
```

Dans le call suspicieux :

```
5-keygen-me-zust-abit-more-difficult.exe - PID: 25824 - Module: 5-keygen-me-zust-abit-more-difficult.exe - Thread: Main Thread 38600 - x64dbg
File View Debug Tracing Plugins Favourites Options Help Mar 15 2025 (TitanEngine)
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles i3 Trace
000000000040156E 55 push rbp
000000000040156F 48 89E5 mov rbp, rsp
0000000000401572 B9 1E000000 mov ecx, 1E
0000000000401578 E8 9B160000 call <MP.&malloc>
000000000040157D 48 8945 F8 mov rax, qword ptr ss:[rbp-8], rax
0000000000401583 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
0000000000401585 C700 5A5A5A mov dword ptr ds:[rax], 5A5A5A5A
0000000000401588 C640 04 00 mov byte ptr ds:[rax+4], 0
000000000040158B 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
000000000040158E C600 42 mov byte ptr ds:[rax], 42
0000000000401596 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
0000000000401599 48 83C0 01 add rax, 1
000000000040159E C600 61 mov byte ptr ds:[rax], 61
00000000004015A1 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015A5 48 83C0 02 add rax, 2
00000000004015A9 C600 62 mov byte ptr ds:[rax], 62
00000000004015AC 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015B0 48 83C0 03 add rax, 3
00000000004015B4 C600 79 mov byte ptr ds:[rax], 79
00000000004015B7 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015B9 48 83C0 04 add rax, 4
00000000004015BF C600 48 mov byte ptr ds:[rax], 48
00000000004015C2 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015C6 48 83C0 05 add rax, 5
00000000004015CA C600 65 mov byte ptr ds:[rax], 65
00000000004015CD 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015D1 48 83C0 06 add rax, 6
00000000004015D5 C600 79 mov byte ptr ds:[rax], 79
00000000004015D8 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015DC 48 83C0 07 add rax, 7
00000000004015E0 C600 67 mov byte ptr ds:[rax], 67
00000000004015E3 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015E7 48 83C0 08 add rax, 8
00000000004015EB C600 65 mov byte ptr ds:[rax], 65
00000000004015F1 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015F2 48 83C0 09 add rax, 9
00000000004015F6 C600 6E mov byte ptr ds:[rax], 6E
00000000004015F9 48 8B45 F8 mov rax, qword ptr ss:[rbp-8]
00000000004015FD 48 83C0 0A add rax, A
7: rbp=00000000004015F0
```

```
...text:0000000000040156B 5-keygen-me-zust-abit-more-difficult.exe:5156B #96B
```

Maintenant si je rentre t000-4444 :

OK maintenant on va coder un keygen :

Le programme regarde le premier index et le 4eme et vérifie que ce soit bien les caractères t et - présent.
Donc il faut que notre keygen génère ces symboles.
Le programme vérifie aussi que la clé soit de taille 9. Si en dessous ou au-dessus ça ne passe pas.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>
/*Le programme regarde le premier index et le 4eme et
vérifie que ce soit bien les caractères t et -
présent.
Donc il faut que notre keygen génère ces symboles.
Le programme vérifie aussi que la clé soit de taille 9.
Si en dessous ou au-dessus ça ne passe pas.
*/

void keygen(){
char first_letter = 't';
char fourth_letter = '-';
char output [10] = {"KEYGENKEY"};
int l = strlen(output) - 1;
srand(time(NULL));
//printf("string avant : %s\n", output);
for (int i = 0; i < l; i++){
output[i] = 'A' + rand() % 26;
//printf("apres la boucle : %s\n", output);
}
output[0] = first_letter;
output[4] = fourth_letter;
printf("Here's your keygen : %s\n", output);
}

int main (){
keygen();
return 0;
}
```